PUFFINLESS

Persistent Memory & Log Storage Engineering Review

Read-only repository review • 8 November 2026

Repository	BvmbinoWorldShop/PuffinLess
Branch / commit	main • d089162be87bf96fa7fdeadaac7df5c2bb5a97ed
Deployment signal	Vercel status reported as success; this does not prove persistence tests passed.

Executive finding

The current storage design is not yet safe enough for irreplaceable daily logs. PuffinLess has useful foundations—encrypted IndexedDB, a localStorage mirror, recovery scanning, and rescue snapshots—but the snapshot is mutable, writes replace whole arrays, and important save operations are launched without awaiting or surfacing failure. A stale or empty in-memory array can therefore replace a more complete copy. The latest commit improves read-error classification, but it does not create an append-only history or independent verified backups.

No browser-based system can honestly promise “never fail.” Browser storage can be cleared, a device can be lost, a browser profile can be corrupted, or encryption keys can become unavailable. The engineering target should be **no silent loss, recoverable history, verified backups, and visible failure states**.

1. What the code currently does

• Data records are encrypted with Web Crypto and stored in IndexedDB via `idbSet()` in `src/utils/indexedDb.ts`.
• Save functions also write encrypted payloads to localStorage.
• `createDeviceRescueSnapshot()` writes a JSON snapshot to localStorage.
• Substance logs, cravings, streaks, sleep/energy, expenses, goals, and profile are persisted through save functions in `src/utils/storage.ts`.

2. Findings that can explain missing week logs

P1 — Rescue snapshot is replaceable, not immutable

The snapshot merges incoming fields with the previous snapshot, then writes the whole object back to one localStorage key. Empty arrays are truthy in JavaScript, so an empty or stale array can replace previously complete history. The implementation is not append-only or immutable despite the code comment.

P1 — Log writes replace entire arrays and are not serialized

Functions such as `saveSubstanceLogs()`, `saveCravings()`, `saveStreaks()`, and `saveSleepEnergy()` encrypt and replace whole arrays. In `useAppStore.ts`, calls are started without awaiting their promises. Concurrent saves can finish out of order, allowing an older state to overwrite a newer one. React handlers also build arrays from captured state values, which can be stale during rapid actions.

P1 — UI success is not tied to durable commit

The UI updates state and starts persistence, but does not consistently wait for a confirmed IndexedDB transaction and read-back. Quota, transaction, encryption, or key failures can leave the screen looking saved even though durable persistence failed.

P1 — Encryption-key persistence still has a gap

At the reviewed commit, `saveKeyToVault()` still returns without error when IndexedDB is unavailable. Its read-back check verifies only that a value exists, not that the retrieved key can perform the expected AES-GCM encrypt/decrypt operation. Legacy migration can therefore proceed under a false assumption of durable key storage.

P2 — Local redundancy is not an independent backup

IndexedDB and localStorage live inside the same browser profile/origin. Clearing site data, losing the device, or profile corruption can remove both. The rescue snapshot is also localStorage and is currently plaintext JSON, creating durability and privacy concerns.

P2 — Recovery is scan-based, not a complete event journal

Recovery can merge records it still finds, but cannot recreate events overwritten or erased from every local copy. A snapshot of current state is not equivalent to a complete history.

3. Recommended architecture — keep the app intact

Keep the current React screens, components, store API, data types, and visual design. Strengthen the persistence layer behind existing save/load functions instead of rebuilding the app.

A. Append-only event journal

Store each meaningful user event as its own immutable IndexedDB record in a dedicated `events` object store. Each record should have a unique ID, monotonic sequence, event type, timestamp, schema version, payload, and integrity metadata. Corrections/deletions become explicit compensating events rather than erasing original events. Existing arrays become projections rebuilt from the journal.

B. Atomic transactions and serialized writes

Commit the event and its sequence/checkpoint metadata in one IndexedDB read-write transaction. Serialize writes through one persistence queue. Acknowledge “Saved” only after transaction completion and read-back verification. Use idempotency keys so retries cannot duplicate events.

C. Versioned checkpoints; preserve the previous good copy

Create numbered checkpoints after a configurable number of events and retain multiple generations. Validate a new checkpoint before marking it current. Keep the previous verified checkpoint until the new one is confirmed. Never replace verified non-empty history with an empty array unless the user explicitly confirms a destructive reset.

D. Fail-safe key lifecycle

If IndexedDB or key-vault persistence is unavailable, preserve legacy key material and stop any migration that would delete it. Verify key algorithm, usages, extractability expectations, and a test encrypt/decrypt round trip. Do not silently continue as though durable storage is available.

E. Local-first external backup under user control

Keep cloud storage off by default. Add an encrypted export/backup file the user saves to a chosen location, plus restore/import with checksum and schema validation. Show the last successful backup time and warn when stale. Browser-local copies alone cannot protect against loss of the whole browser profile or device.

F. Visible persistence state

Expose Saving, Saved, Backup needed, Storage unavailable, and Recovery required states. On write failure, retain the pending event in memory, show the failure, retry safely, and do not claim the event is durably saved. Startup should enter recovery/read-only mode if integrity checks fail instead of silently substituting empty defaults.

4. Immediate safety rules

• Do not clear browser data, uninstall, reset, or reinstall before attempting recovery.
• Do not run `resetAllUserData()` while recovering; it intentionally deletes the database and keys.
• Preserve the current browser profile and device. Recovery may still be possible if an IndexedDB record or rescue snapshot remains.
• Do not claim the missing week can be restored until the actual browser storage has been inspected. If every local copy was erased and no export exists, code changes cannot recreate those events.

5. Verification required before release

This review inspected the repository but did not execute tests, build, lint, or a real browser cold-restart scenario. A Vercel “success” status is not proof that data persistence and recovery tests passed.

Test	Required pass condition
Rapid concurrent logging	All events survive; no older write overwrites newer events.
Refresh / tab close / browser restart	Full history rehydrates with exact IDs and timestamps.
IndexedDB blocked / quota / abort	No false “Saved”; pending event remains recoverable; error shown.

Test	Required pass condition
Key-vault unavailable during migration	Legacy key is not deleted; durable persistence is not falsely claimed.
Corrupt latest checkpoint	Previous verified checkpoint plus journal replay restores history.
Empty state / reset	No automatic empty-array overwrite; reset requires explicit confirmation.
Export then restore on clean profile	All events and projections match checksums and counts.
Multi-tab / rapid actions	Sequence stays unique; writes are coordinated; no lost updates.

6. Scope and current status

Repository changes made during this review: none. The latest visible main-branch source at commit d089162 was inspected. No files were modified, no commit was pushed, no pull request was opened, and no deployment was triggered. Vercel status was reported successful; tests were not independently executed.

Can the deleted week be recovered? Possibly, but only if a surviving copy exists in IndexedDB, the local rescue snapshot, a browser-profile backup, or a prior export. This repository review cannot inspect the user's device storage. Do not reset or clear the app while recovery is still being investigated.

Recommended implementation order

1) Preserve and inspect remaining local data. 2) Fix key-vault failure handling and serialize/await saves. 3) Add append-only event journal and versioned checkpoints. 4) Add encrypted user-controlled backup/restore. 5) Add browser-level failure and cold-restart tests. 6) Release only after all checks pass.

Files examined: src/utils/crypto.ts, src/utils/indexedDb.ts, src/utils/storage.ts, src/utils/recovery.ts, src/state/useAppStore.ts, src/App.tsx, tests/engineering-verification.test.js, package.json.

Engineering assessment only; not a guarantee of recovery or zero-failure operation.